Cell id ⇒ unique cell no.
lot ⇒ production batch
test_rig ⇒ which test machine tested the cell

We have this for 0,1,..
- V_cycleN Voltage in volts
- R_int_cycleN Internal Resistance in mili ohms mΩ
- C_cycleN discharge capacity in Amp hours Ah
- T_cycleN cell Temperature in °C

We have to accurately predict this
- C_cycle200 Capacity of cell at cycle 200
- Retention_cycle200 = $\frac{C\_cycle200}{C\_cycle0}$ = To normalise the result
- is_defective 0 or 1
- defect_type resistance_drift, capacity_only_fade, Thermal_runaway_Tendancy, Voltage_sag, none.

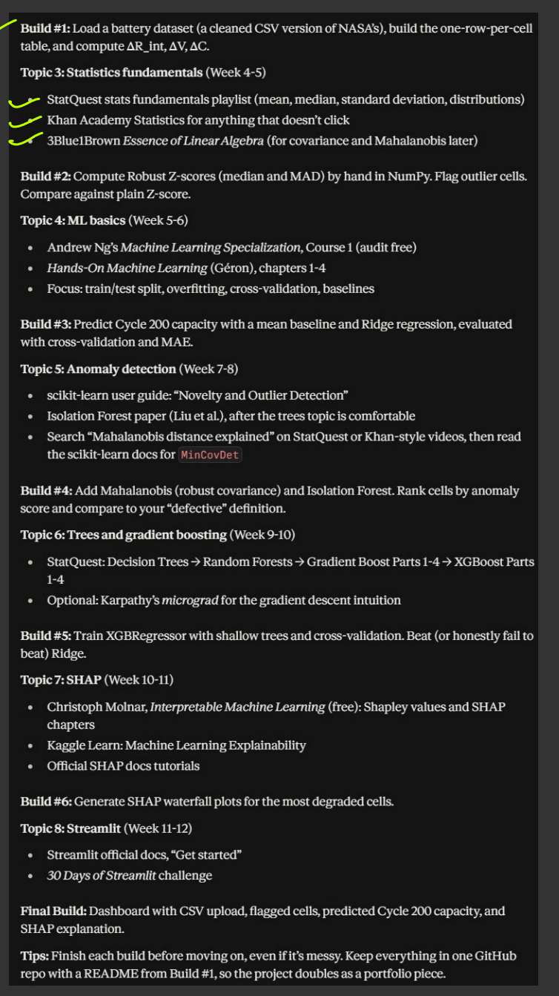
$V = IR$
$V_{terminal} = V_{open\ circuit} - I R_{internal}$
$P = I^2 R$
$C = It$
C_retention = $\frac{C \text{ at } N}{C \text{ at } 0}$
$\Delta R = R_{cycle N} - R_{cycle 0}$
ΔV
ΔC
ΔT
State of Health (SOH) = When Retention is 80% battery is at the end of its life

Available cycles:- 0, 5, 10, 25, 50, 100, 200

(I) use 0, 5, 10 to predict 200

(II) use 0, 5, 10, 25, 50, 100 to predict 200

Plan 1



✓ **Build #1:** Load a battery dataset (a cleaned CSV version of NASA's), build the one-row-per-cell table, and compute ΔR_int, ΔV, ΔC.

Topic 3: Statistics fundamentals (Week 4-5)

- ✓ StatQuest stats fundamentals playlist (mean, median, standard deviation, distributions)
- ✓ Khan Academy Statistics for anything that doesn't click
- ✓ 3Blue1Brown *Essence of Linear Algebra* (for covariance and Mahalanobis later)

Build #2: Compute Robust Z-scores (median and MAD) by hand in NumPy. Flag outlier cells. Compare against plain Z-score.

Topic 4: ML basics (Week 5-6)

- Andrew Ng's *Machine Learning Specialization*, Course 1 (audit free)
- *Hands-On Machine Learning* (Géron), chapters 1-4
- Focus: train/test split, overfitting, cross-validation, baselines

Build #3: Predict Cycle 200 capacity with a mean baseline and Ridge regression, evaluated with cross-validation and MAE.

Topic 5: Anomaly detection (Week 7-8)

- scikit-learn user guide: "Novelty and Outlier Detection"
- Isolation Forest paper (Liu et al.), after the trees topic is comfortable
- Search "Mahalanobis distance explained" on StatQuest or Khan-style videos, then read the scikit-learn docs for `MinCovDet`

Build #4: Add Mahalanobis (robust covariance) and Isolation Forest. Rank cells by anomaly score and compare to your "defective" definition.

Topic 6: Trees and gradient boosting (Week 9-10)

- StatQuest: Decision Trees → Random Forests → Gradient Boost Parts 1-4 → XGBoost Parts 1-4
- Optional: Karpathy's *micrograd* for the gradient descent intuition

Build #5: Train XGBRegressor with shallow trees and cross-validation. Beat (or honestly fail to beat) Ridge.

Topic 7: SHAP (Week 10-11)

- Christoph Molnar, *Interpretable Machine Learning* (free): Shapley values and SHAP chapters
- Kaggle Learn: Machine Learning Explainability
- Official SHAP docs tutorials

Build #6: Generate SHAP waterfall plots for the most degraded cells.

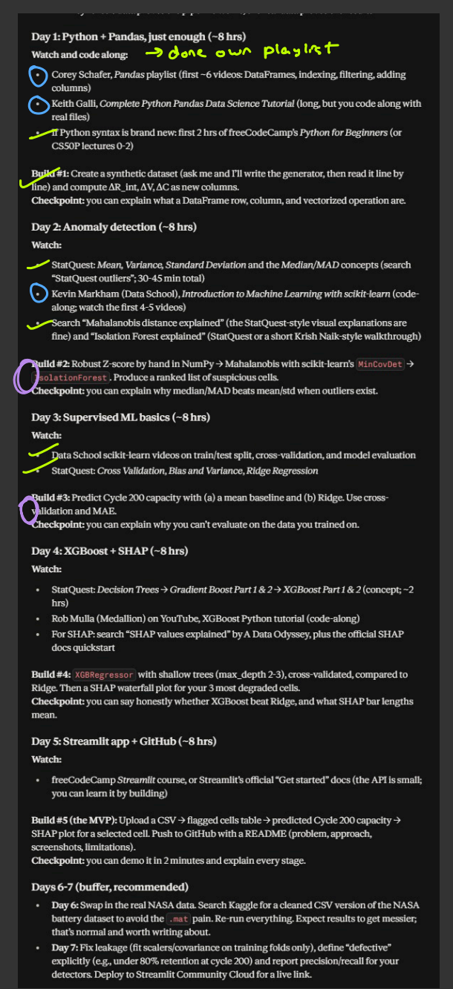
Topic 8: Streamlit (Week 11-12)

- Streamlit official docs, "Get started"
- *30 Days of Streamlit* challenge

Final Build: Dashboard with CSV upload, flagged cells, predicted Cycle 200 capacity, and SHAP explanation.

Tips: Finish each build before moving on, even if it's messy. Keep everything in one GitHub repo with a README from Build #1, so the project doubles as a portfolio piece.

Plan 2



Day 1: Python + Pandas, just enough (~8 hrs)

Watch and code along: → done own playlist

- Corey Schafer, *Pandas* playlist (first ~6 videos: DataFrames, indexing, filtering, adding columns)
- Keith Galli, *Complete Python Pandas Data Science Tutorial* (long, but you code along with real files)
- ✓ If Python syntax is brand new: first 2 hrs of freeCodeCamp's *Python for Beginners* (or CS50P lectures 0-2)

✓ **Build #1:** Create a synthetic dataset (ask me and I'll write the generator, then read it line by line) and compute ΔR_int, ΔV, ΔC as new columns.
Checkpoint: you can explain what a DataFrame row, column, and vectorized operation are.

Day 2: Anomaly detection (~8 hrs)

Watch:

- ✓ StatQuest: *Mean, Variance, Standard Deviation* and the *Median/MAD* concepts (search "StatQuest outliers"; 30-45 min total)
- Kevin Markham (Data School), *Introduction to Machine Learning with scikit-learn* (code-along; watch the first 4-5 videos)
- ✓ Search "Mahalanobis distance explained" (the StatQuest-style visual explanations are fine) and "Isolation Forest explained" (StatQuest or a short Krish Naik-style walkthrough)

Build #2: Robust Z-score by hand in NumPy → Mahalanobis with scikit-learn's `MinCovDet` → `IsolationForest`. Produce a ranked list of suspicious cells.
Checkpoint: you can explain why median/MAD beats mean/std when outliers exist.

Day 3: Supervised ML basics (~8 hrs)

Watch:

- ✓ Data School scikit-learn videos on train/test split, cross-validation, and model evaluation
- ✓ StatQuest: *Cross Validation, Bias and Variance, Ridge Regression*

Build #3: Predict Cycle 200 capacity with (a) a mean baseline and (b) Ridge. Use cross-validation and MAE.
Checkpoint: you can explain why you can't evaluate on the data you trained on.

Day 4: XGBoost + SHAP (~8 hrs)

Watch:

- StatQuest: *Decision Trees* → *Gradient Boost Part 1 & 2* → *XGBoost Part 1 & 2* (concept; ~2 hrs)
- Rob Mulla (Medallion) on YouTube, XGBoost Python tutorial (code-along)
- For SHAP: search "SHAP values explained" by A Data Odyssey, plus the official SHAP docs quickstart

Build #4: `XGBRegressor` with shallow trees (max_depth 2-3), cross-validated, compared to Ridge. Then a SHAP waterfall plot for your 3 most degraded cells.
Checkpoint: you can say honestly whether XGBoost beat Ridge, and what SHAP bar lengths mean.

Day 5: Streamlit app + GitHub (~8 hrs)

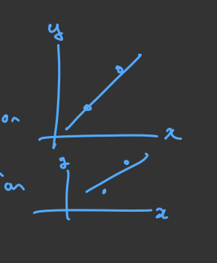
Watch:

- freeCodeCamp *Streamlit* course, or Streamlit's official "Get started" docs (the API is small; you can learn it by building)

Build #5 (the MVP): Upload a CSV → flagged cells table → predicted Cycle 200 capacity → SHAP plot for a selected cell. Push to GitHub with a README (problem, approach, screenshots, limitations).
Checkpoint: you can demo it in 2 minutes and explain every stage.

Days 6-7 (buffer, recommended)

- **Day 6:** Swap in the real NASA data. Search Kaggle for a cleaned CSV version of the NASA battery dataset to avoid the `.mat` pain. Re-run everything. Expect results to get messier; that's normal and worth writing about.
- **Day 7:** Fix leakage (fit scalers/covariance on training folds only), define "defective" explicitly (e.g., under 80% retention at cycle 200) and report precision/recall for your detectors. Deploy to Streamlit Community Cloud for a live link.



linear regression

ridge regression

$y = mx + c$ ⇒ line of least square fit

WE HAVE TO MINIMISE ⇒ Sum of the squared residuals + $\lambda \cdot m^2$

$\lambda \to [0, \infty)$

Actual slope of line from training Data

Bias we add to slightly tip the data to our favour

Residual = Actual Value − Predicted Value

eg:- we have 2 training pts
$(x, y) \Rightarrow (1,3), (2,4)$
$m = 1$ intercept $= 2$
$y = x + 2$

What the algo does ⇒
I starts Varying lambda

If we have 2 points $(x_1, y_1), (x_2, y_2)$
$L(m, c) = \sum (\text{residual})^2 + \lambda m^2$
$L(m, c) = (y_1 - y_{1\,expected})^2 + (y_2 - y_{2\,expected})^2 + \lambda m^2$
$L(m, c) = (y_1 - mx_1 - c)^2 + (y_2 - mx_2 - c)^2 + \lambda m^2$
↳ minimise $= \alpha m^2 + \beta m + \gamma$ ⇒ minimised $m = m'$
to get y-intercept ⇒ $c = \text{avg}(y) - (\text{minimised } m) \cdot \text{avg}(x)$
$= \bar{y} - m'\bar{x}$

If I have (n+1) parameters [(n+1) dimension graph] ⇒ $L(m_1, m_2, \dots m_n, c) = \sum (\text{residual})^2 + \lambda \sum_{i=1}^{n} m_i$

⇒ Parameter1 = m_1 Parameter$_2$ + m_2 Parameter 3 + … + y-intercept

RIDGE REGRESSION

instead of γm^2 ⇒ if we use $\lambda |m|$

XGBoost ⇒ Extreme Gradient Boost
- ⇒ Involves both L1 & L2 Regulation
- ⇒ Handles missing values
- ⇒ Built in Cross-Validation
- ⇒

GRADIENT BOOST

Gradient Boost is Regression version of Linear Regression in Classification

How to find Anomalies?

Probabilistic / Statistical Approach
- Benford's law
- Z-scores
- Mahalanobis Distance

Machine learning Approach
- k-Nearest Neighbours
- Local Outlier Factor
- Isolation Forest
- Classifier Adjusted Density Estimation (CADE)
- One class SVM

} For much larger factors

① PANDAS
- ✓ → Brocode
- → Corey Schafer (first 6 videos)
- → Keith Galli (1 big video)

② Math Concepts Required
- → Mean = $\frac{\sum x}{n}$ $E(x) = \sum x\,p(x)$
- → Variance = σ^2
- → Standard deviation ⇒ $\sqrt{E(x^2) - (E(x))^2} = \sigma$ ⇒ $\sqrt{\frac{\sum (x_i - \bar{x})^2}{n-1}}$

③ Mean Absolute deviation = $\frac{\sum |x_i - \bar{x}|}{n}$ MAD ≤ σ

④ MAD (Median Absolute Deviation) → Problem with σ is that they are heavily impacted by outliers = median $(x_i - \text{median}(x))$

Single extreme value can affect judgement more than it should

When to use:-
- Data has extreme outliers
- when dist. is not normal or symmetric
- when comparing datasets with diff. scales

When to use what?

① Standard Deviation — Data is symmetric with no outliers → when we build statistical models [linear regression, z-scores, hypothesis testing (t-tests, ANOVA)]

[My extreme outcome is a result of common behavior and we want to take that into account]

② Mean Absolute Deviation — [simple forcasting models, no real use]

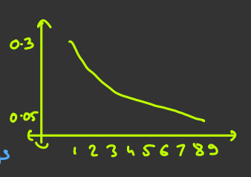
③ Median Absolute Deviation — Data is skewed or has many outliers → When we are looking at data whose extremes are outliers and not a result of data we want to take into account

STATISTICAL APPROACH FOR CLASSIFICATION

① Benford's law (For entities that ⓐ large sample size ⓑ Arrises from a multiplicative process)

if d_n is the leading digit of any number
10368 ⇒ 1
203 ⇒ 2
9832186 ⇒ 9

Then in a given dataset $P(d_n) = \log_{10}\left(1 + \frac{1}{d_n}\right)$



Why?
Idea:- Because in the real world, things grow exponentially
1000 → 2000 → 4000 → 8000 → 16000 (1 year, 1 year, 1 year, 1 year)
1000 → 2000 → 3000 → 4000 .. (1 year, ½ year, ⅓ year)

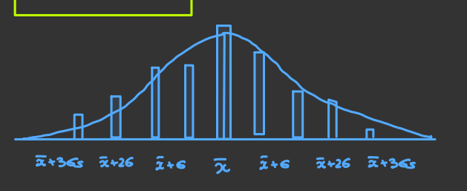
Second logic:- you look at a thing that goes from 1 → N
N = 1, 2, 3, 4, 5, 6, 7, 8, 9
[in every case we have a 1 included but only 1 case, we have a 9.]

① Used for fraud detection
② Used for detection of investment return data for Ponzi schemes
- χ^2 test used to compare observed dist. to expected dist. under Benford's law

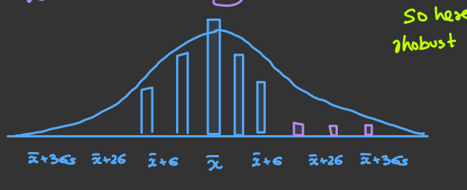
② Z-scores (used when dist. is normal)

$$\boxed{z_i = \frac{x_i - \bar{x}}{\sigma}}$$

how many standard deviations away the pt. is from the mean



what if anomaly



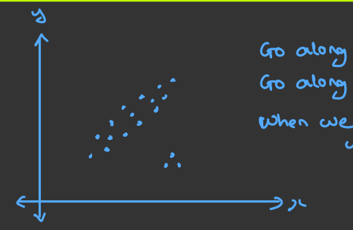
So here we use robust z-score

$$\boxed{z = \frac{x_i - \text{median}(x)}{\text{MAD}(x)}}$$

MAD = k · median (|xi − median(xi)|)
for normal dist k = 1.48
↳ Adjustment factor

now, Anomalies are more than 3 MAD away from medians

③ MAHALANOBIS DISTANCE



Go along x ⇒ no outliers
Go along y ⇒ no outliers
when we consider both x, y, we see outliers

for the robust version, we isolate the h tightest points and calculate μ & Σ as μ_{MCD}, Σ_{MCD}

Mahalanobis Distence $D_M = \sqrt{(x - \mu)^T \Sigma^{-1} (x - \mu)}$

here, x is a position vector
μ mean vector
Σ covariance matrix
Σ^{-1} is its inverse

Covariance Matrix = $\begin{bmatrix} \text{variance of } x & \text{covariance btw } x, y \\ \text{covariance btw } x, y & \text{variance of } y \end{bmatrix}$

Covariance = Correlation × $\sigma_x \times \sigma_y$

Correlation = $\frac{\sigma_{xy}}{\sigma_x \sigma_y}$

Robust Version $D_M = \sqrt{(x - \mu_{MCD})^T \Sigma_{MCD}^{-1} (x - \mu_{MCD})}$

Then we get D_M for every point

⇒ If my dataset is roughly normal, $(D_M)^2$ follows a Chi-Squared dist. (χ^2) with p-degrees of freedom where p is the number of features

OUTLIER CONDITION :- $\boxed{D_M^2 > \chi^2_{critical}(p, 1-\alpha)}$
where α ⇒ cutoff percentage
eg:- if α = 0.01
Points in the top 1% are outliers and 99% points are safe

∴ The higher the D_M the more likely it is to be an outlier

Mahalanobis distance
(Q) We know for [height, weight]
μ = [170cm, 70kg]
Standard Deviation is 10 for both
Correlation = 0.6 Correlation = $\frac{\sigma_{xy}}{\sigma_x \sigma_y}$

Covariance = $\Sigma = \begin{bmatrix} 100 & 60 \\ 60 & 100 \end{bmatrix}$

$\Sigma^{-1} = \frac{1}{6400} \begin{bmatrix} 100 & -60 \\ -60 & 100 \end{bmatrix}$

NOW USE FORMULA
$DM^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$
let $(x - \mu) = [p, q]$
⇒ $DM^2 = \frac{100p^2 - 120pq + 100q^2}{6400}$

NOW WE CONSIDER 2 PTS :- (190, 90), (190, 50)
Person A:- p = 20, q = −20 ⇒ DM = 2.25
Person B:- p = 20, q = −20 ⇒ DM = 4.47

if DM > 2.72 ⇒ Outlier

MACHINE LEARNING APPROACH FOR CLASSIFICATION

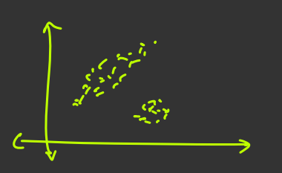
① K Nearest Neighbours
Instead of measuring distance from some sort of centre
We measure average distance of a point from its nearest k neighbours
- Euclidean distance
- Mahalanobis distance

problem:- Change in scale changes distance
To solve:- Standardisation required

Then make a dist. of all scores and for score > [median + 3MAD] } outlier
only check on the (+ve) side
if it is less than 3MAD it means data is dense & that is OK → Wouldn't this cause overfitting??

What if Density = $\frac{1}{\text{Avg of k nearest neighbours}}$

(*) limitation
⇒ We have 2 clusters



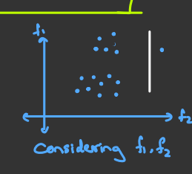
Then in certain cases, there are no outliers

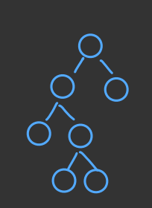
② Local Outlier Factor (LOF)
ratio R = $\frac{\text{avg. density of a pt.}}{\text{avg. density of its k-nearest neighbours}}$
R > 1 ⇒ likely to be Anomaly
R < 1 ⇒ less likely to be Anomaly

Density = $\frac{1}{\text{Reachability}}$
Reachability = Distance between the point and its nearest neighbour → Clarify this

Density ⇒ Area of circle ⇒ centre ⇒ point
radius ⇒ centre to the k^{th} closesest point

③ ISOLATION FOREST [decision Trees]
f_1 f_2 f_3 f_4 … features





Anomaly Score
N → no. of datapoints
x → particular data pt
E(h(x)) → Average search depth for x from isolated tree
C(n) → Average dept of all data points

$\boxed{\text{Max Depth} = \log_2(N)}$

$\boxed{C(n) = 2H(n-1) - \frac{2(n-1)}{n}}$ (do not approximate for small values)

$\boxed{H(i) = 1 + \frac{1}{2} + \dots + \frac{1}{i} = \ln(i) + 0.5772156649 \approx \ln(i) + 0.5772}$

$\boxed{\text{Score}(x, n) = 2^{-\frac{\text{average of scores for that } x}{C(n)}}}$

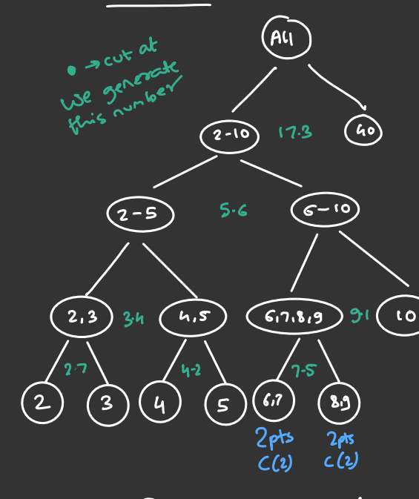
Score close to 1 ⇒ Outlier
around 0.5 ⇒ Average (nothing stands out)
0.5 ⇒ Very Normal
very less than 0.5 ⇒ No real outliers in data

Generally, we are supposed to have 100 trees

eg:- Data :- 2, 3, 4, 5, 6, 7, 8, 9, 10, 40
ψ = 10 ⇒ Total 10 pts.
max depth = ceil (log₂10) = ceil. (3.xx) = 4
$C(\psi) = 2 \cdot (\ln 9 + 0.5772) - \frac{2 \cdot 9}{10} = 3.75$

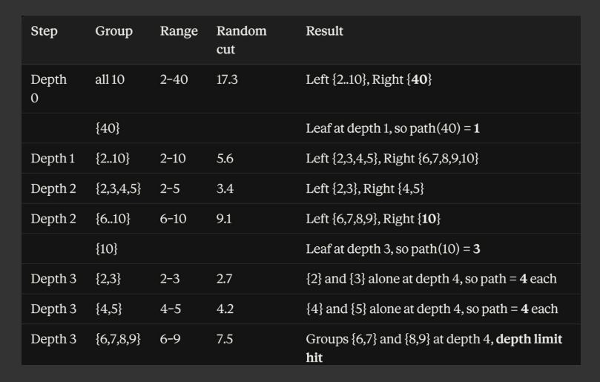
Leaf Corrections (needed if branch stops at depth 4 with several points) = C(1) = 0
C(2) = 1
C(3) = 2 (ln2 + 0.5772) − $\frac{4}{3}$ But n is small, no approximation here = $2\left(1 + \frac{1}{2}\right) - \frac{4}{3} = 1.67$
C(4) = ---
C(5) = ---

TREE 1



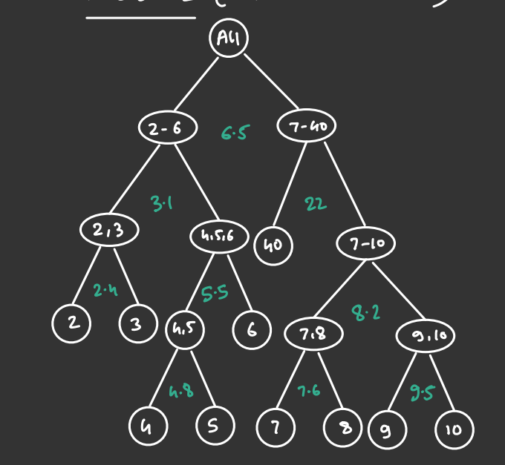
Depth 0 / Depth 1 / Depth 2 / Depth 3 / Depth 4

paths for tree 1 it took to get to that number:
2 → 4
3 → 4
4 → 4
5 → 4
6 → 4 + c(2) = 4 + 1 = 5
7 → 5
8 → 5
9 → 5
10 → 3
40 → 1



Step	Group	Range	Random cut	Result
Depth 0	all 10	2–40	17.3	Left {2..10}, Right {40}
	{40}			Leaf at depth 1, so path(40) = **1**
Depth 1	{2..10}	2–10	5.6	Left {2,3,4,5}, Right {6,7,8,9,10}
Depth 2	{2,3,4,5}	2–5	3.4	Left {2,3}, Right {4,5}
Depth 2	{6..10}	6–10	9.1	Left {6,7,8,9}, Right {10}
	{10}			Leaf at depth 3, so path(10) = **3**
Depth 3	{2,3}	2–3	2.7	{2} and {3} alone at depth 4, so path = **4** each
Depth 3	{4,5}	4–5	4.2	{4} and {5} alone at depth 4, so path = **4** each
Depth 3	{6,7,8,9}	6–9	7.5	Groups {6,7} and {8,9} at depth 4, **depth limit hit**

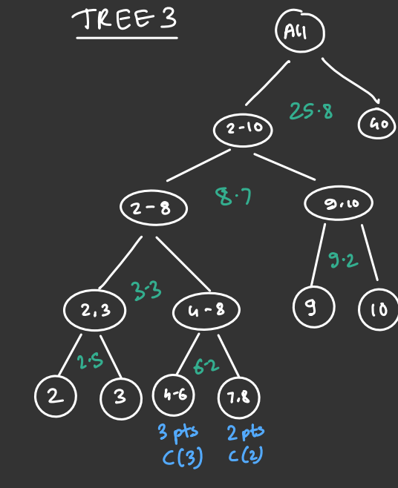
TREE 2 (unfortunate first cut)



Depth 0 / Depth 1 / Depth 2 / Depth 3 / Depth 4

paths for tree 2 it took to get to that number:
2 → 3
3 → 3
4 → 4
5 → 4
6 → 3
7 → 4
8 → 4
9 → 4
10 → 4
40 → 4

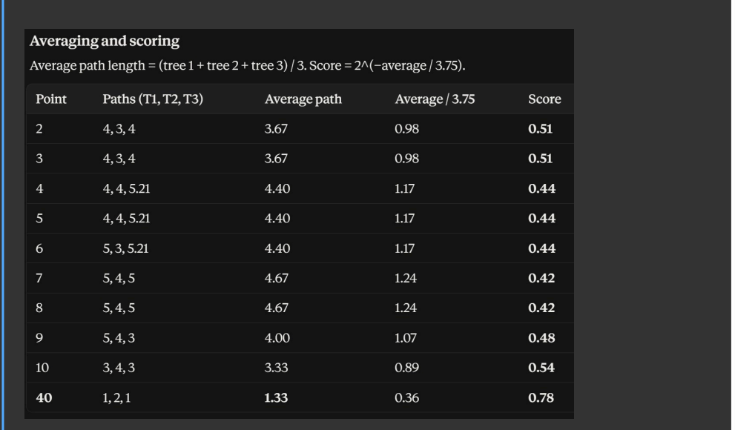
TREE 3



Depth 0 / Depth 1 / Depth 2 / Depth 3 / Depth 4

paths for tree 2 it took to get to that number:
2 → 4
3 → 4
4 → 4 + C(3) = 4 + 1.21 = 5.21
5 → 5.21
6 → 5.21
7 → 5
8 → 5
9 → 3
10 → 3
40 → 1

AVERAGING AND SCORING



Averaging and scoring

Average path length = (tree 1 + tree 2 + tree 3) / 3. Score = 2^(−average / 3.75).

Point	Paths (T1, T2, T3)	Average path	Average / 3.75	Score
2	4, 3, 4	3.67	0.98	**0.51**
3	4, 3, 4	3.67	0.98	**0.51**
4	4, 4, 5.21	4.40	1.17	**0.44**
5	4, 4, 5.21	4.40	1.17	**0.44**
6	5, 3, 5.21	4.40	1.17	**0.44**
7	5, 4, 5	4.67	1.24	**0.42**
8	5, 4, 5	4.67	1.24	**0.42**
9	5, 4, 3	4.00	1.07	**0.48**
10	3, 4, 3	3.33	0.89	**0.54**
40	1, 2, 1	**1.33**	0.36	**0.78**

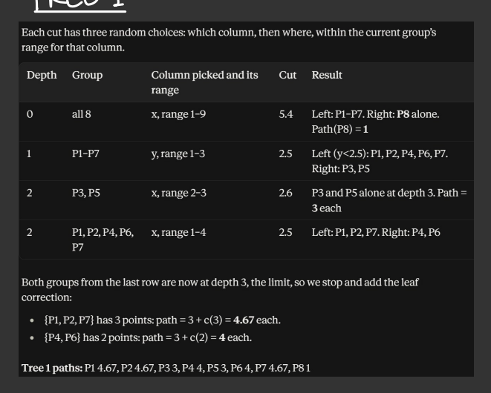
Score of 40 = 0.78 → Outlier point

Edge Effect ⇒ 2, 3, 10 have a relatively higher score ⇒ They are at the end

If 2 features
P1 (1,2)
P2 (2,1)
P3 (3,2)
P4 (2,3)
P5 (3,3)
P6 (4,2)
P7 (2,2)
P8 (9,8)

N = 8
Max depth = ceil(log₂(8)) = 3
C(1) = 0, C(2) = 1, C(3) = 1.67, C(8) = 3.44

TREE 1



Each cut has three random choices: which column, then where, within the current group's range for that column.

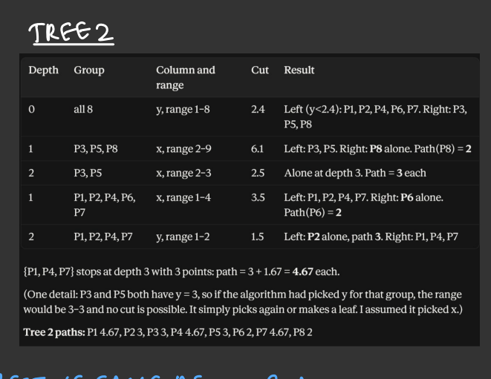
Depth	Group	Column picked and its range	Cut	Result
0	all 8	x, range 1-9	5.4	Left: P1–P7. Right: **P8** alone. Path(P8) = **1**
1	P1–P7	y, range 1-3	2.5	Left (y<2.5): P1, P2, P4, P6, P7. Right: P3, P5
2	P3, P5	x, range 2-3	2.6	P3 and P5 alone at depth 3. Path = **3** each
2	P1, P2, P4, P6, P7	x, range 1-4	2.5	Left: P1, P2, P7. Right: P4, P6

Both groups from the last row are now at depth 3, the limit, so we stop and add the leaf correction:

- {P1, P2, P7} has 3 points: path = 3 + c(3) = **4.67** each.
- {P4, P6} has 2 points: path = 3 + c(2) = **4** each.

Tree 1 paths: P1 4.67, P2 4.67, P3 3, P4 4, P5 3, P6 4, P7 4.67, P8 1

TREE 2



Depth	Group	Column and range	Cut	Result
0	all 8	y, range 1-8	2.4	Left (y<2.4): P1, P2, P4, P6, P7. Right: P3, P5, P8
1	P3, P5, P8	x, range 2-9	6.1	Left: P3, P5. Right: **P8** alone. Path(P8) = **2**
2	P3, P5	x, range 2-3	2.5	Alone at depth 3. Path = **3** each
1	P1, P2, P4, P6, P7	x, range 1-4	3.5	Left: P1, P2, P4, P7. Right: **P6** alone. Path(P6) = **2**
2	P1, P2, P4, P7	y, range 1-2	1.5	Left: **P2** alone, path **3**. Right: P1, P4, P7

{P1, P4, P7} stops at depth 3 with 3 points: path = 3 + 1.67 = **4.67** each.

(One detail: P3 and P5 both have y = 3, so if the algorithm had picked y for that group, the range would be 3-3 and no cut is possible. It simply picks again or makes a leaf. I assumed it picked x.)

Tree 2 paths: P1 4.67, P2 3, P3 3, P4 4.67, P5 3, P6 2, P7 4.67, P8 2

REST IS SAME AS 1 feature.

Ml techniques
① Logistic regression
② K-Nearest neighbours
③ SVMs (Support Vector Machines)